MultiGridBarrier.jl: quasi-optimal solvers for convex variational problems

Sébastien Loisel [1]

1 Heriot-Watt University, Edinburgh, United Kingdom

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

`MultiGridBarrier.jl` is a Julia package for solving convex variational problems in function spaces. These are the nonlinear partial differential equations (PDEs) and boundary-value problems that arise from minimizing a convex functional. Representative examples include the p-Laplacian for any $p \in [1, \infty]$, total-variation problems, and obstacle problems. The most useful of these are *nonsmooth*: the energy is convex but not differentiable (e.g. $p = 1$ or total variation), a regime in which Newton-type solvers applied naively either fail or require a number of iterations that grows rapidly with the mesh resolution.

The package implements the **multigrid barrier method**, which couples an interior-point (barrier) method with a multigrid hierarchy. For the problem classes covered by the supporting theory the method is *quasi-optimal*: the number of interior-point/Newton iterations grows only mildly with the number of degrees of freedom n. For instance, this count is $O(\sqrt{n}\,\log n)$ for the p-Laplacian (Loisel, 2020), and polylogarithmic in the analytic, spectral setting (Loisel, 2026).

`MultiGridBarrier.jl` provides finite-element discretizations in one, two, and three dimensions (simplicial P_1/P_2 elements and tensor-product Q_k elements), as well as Chebyshev spectral discretizations, all with isoparametric element maps. It builds an algebraic-multigrid hierarchy automatically (via `AlgebraicMultigrid.jl` (JuliaLinearAlgebra contributors, 2024) or, optionally, `PyAMG` (Bell et al., 2023)), supports user-specified mesh connectivity (enabling slit domains, branch cuts, and glued manifolds), solves time-dependent problems, and offers optional GPU acceleration through CUDA. A typical solve is three lines:

```julia
using MultiGridBarrier
geom = fem2d_P2()
sol  = mgb_solve(assemble(amg(geom); p = 1.0))   # a nonsmooth p = 1 problem
```


Statement of need

Convex variational problems are ubiquitous in computational science: nonlinear elasticity and plasticity, image denoising and segmentation (total variation), contact and obstacle problems, and non-Newtonian flow (the p-Laplacian). The difficulty is that the most interesting cases are nonsmooth (the energy is convex but not differentiable), so Newton-type methods applied naively either stagnate or require an iteration count that grows rapidly as the mesh is refined.

Interior-point (barrier) methods handle nonsmoothness robustly by following a smooth central path, but a single barrier solve still requires solving a sequence of large, increasingly ill-conditioned linear systems. The multigrid barrier method addresses both issues at once: a multigrid hierarchy preconditions the central-path subproblems so that the *total* cost stays close to linear in the number of unknowns, with rigorous bounds for the covered problem classes (Loisel, 2020, 2026).

General-purpose finite-element libraries and algebraic-multigrid libraries in Julia provide the building blocks for discretizing PDEs and solving linear systems, but they do not provide an out-of-the-box, theoretically grounded solver for nonsmooth convex variational problems. `MultiGridBarrier.jl` fills this gap: it packages the discretization, the multigrid hierarchy, and the barrier solver behind a small high-level interface (`fem2d_P2`, `amg`, `assemble`, `mgb_solve`). Researchers and practitioners can then solve such problems, and reproduce the numerical results of the underlying papers, in a few lines on the CPU or the GPU.

Functionality

- **Discretizations.** Finite elements in 1D/2D/3D: simplicial P_1/P_2 and tensor-product Q_k, plus Chebyshev spectral elements; all isoparametric.
- **Solver.** An algebraic-multigrid hierarchy (`amg`) drives a barrier (interior-point) method (`mgb_solve`) for user-assembled convex problems (`assemble`).
- **Convex constraints.** Built-in convex sets for p-norm/Euclidian-power, linear, and piecewise constraints, composable via `intersect`.
- **Topological meshes.** Explicit connectivity (`tensor_dofmap` and the `t=` keyword) lets geometrically coincident nodes remain topologically distinct, supporting slit domains, branch cuts, and glued manifolds.
- **Time dependence.** `parabolic_solve` for time-dependent problems.
- **GPU.** Optional CUDA acceleration through a package extension.
- **Visualization.** Plotting of 1D/2D/3D solutions and animations.

References

Bell, N., Olson, L. N., Schroder, J., & Southworth, B. (2023). *PyAMG: Algebraic multigrid solvers in Python*. https://github.com/pyamg/pyamg.

JuliaLinearAlgebra contributors. (2024). *AlgebraicMultigrid.jl: Algebraic multigrid (AMG) solvers in Julia*. https://github.com/JuliaLinearAlgebra/AlgebraicMultigrid.jl.

Loisel, S. (2020). Efficient algorithms for solving the p-laplacian in polynomial time. *Numerische Mathematik*, *146*(2), 369–400. https://doi.org/10.1007/s00211-020-01141-z

Loisel, S. (2026). The spectral barrier method to solve analytic convex optimization problems in function spaces. *Numerische Mathematik*, *158*(1), 281–302. https://doi.org/10.1007/s00211-025-01508-0